Document number: P1969R0
Date: 2019-11-08
Project: Programming Language C++
Reference: ISO/IEC IS 14882:2017
Reply to: William M. Miller
Edison Design Group, Inc.
wmm@edg.com

Core Language Working Group "ready" Issues for the November, 2019 (Belfast) meeting

References in this document reflect the section and paragraph numbering of document WG21 N4835.

2280. Matching a usual deallocation function with placement new

Section: 7.6.2.7 [expr.new] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-06-27

The restrictions in 7.6.2.7 [expr.new] against a placement deallocation function signature matching a usual deallocation function signature consider only `std::size_t` parameters, omitting `std::align_val_t` parameters.

Proposed resolution (February, 2017):

Change 7.6.2.7 [expr.new] paragraph 27 as follows:

> A declaration of a placement deallocation function matches the declaration of a placement allocation function if it has the same number of parameters and, after parameter transformations (9.3.3.5 [dcl.fct]), all parameter types except the first are identical. If the lookup finds a single matching deallocation function, that function will be called; otherwise, no deallocation function will be called. If the lookup finds a usual deallocation function ~~with a parameter of type `std::size_t` (6.7.5.4.2 [basic.stc.dynamic.deallocation])~~ and that function, considered as a placement deallocation function, would have been selected as a match for the allocation function, the program is ill-formed. For a non-placement allocation function, the normal deallocation function lookup is used to find the matching deallocation function (7.6.2.8 [expr.delete]). [*Example*:...

>

2382. Array allocation overhead for non-allocating placement `new`

Section: 7.6.2.7 [expr.new] **Status:** ready **Submitter:** Paul Sanders **Date:** 2018-07-16

According to 7.6.2.7 [expr.new] paragraph 12,

> When a *new-expression* calls an allocation function and that allocation has not been extended, the *new-expression* passes the amount of space requested to the allocation function as the first argument of type std::size_t . That argument shall be no less than the size of the object being created; it may be greater than the size of the object being created only if the object is an array. For arrays of `char`, `unsigned char`, and `std::byte`, the difference between the result of the *new-expression* and the

address returned by the allocation function shall be an integral multiple of the strictest fundamental alignment requirement (6.7.6 [basic.align]) of any object type whose size is no greater than the size of the array being created.

There is no exemption for the non-allocating `(void*,size_t)` placement-new allocation function, so programs must allow for the possibility that the provided buffer may need to be larger (by an indeterminate amount) than the size of an array placed into existing storage.

Should the non-allocating placement-new allocation function be exempt from the array allocation overhead? (This question was explicitly referred to CWG by the EWG chair.)

Proposed resolution (February, 2019):

1. Change 7.6.2.7 [expr.new] paragraph 12 as follows:

> When a *new-expression* calls an allocation function and that allocation has not been extended, the *new-expression* passes the amount of space requested to the allocation function as the first argument of type std::size_t . That argument shall be no less than the size of the object being created; it may be greater than the size of the object being created only if the object is an array **and the allocation function is not a non-allocating form (17.6.2.3 [new.delete.placement])**. For arrays of...

2. Change 7.6.2.7 [expr.new] paragraph 16 as follows:

> ...Here, each instance of `x` is a non-negative unspecified value representing array allocation overhead; the result of the *new-expression* will be offset by this amount from the value returned by `operator new[]`. This overhead may be applied in all array *new-expression*s, including those referencing **a placement allocation function, but not when referencing** the library function `operator new[](std::size_t, void*)` ~~and other placement allocation functions~~. The amount of overhead may vary from one invocation of `new` to another. —*end example*]

(This resolution effectively resolves issue 476, which was closed for EWG input.)

2416. Explicit specializations vs `constexpr` and `consteval`

Section: 13.9.3 [temp.expl.spec] **Status:** ready **Submitter:** Mike Miller **Date:** 2019-06-07

According to 13.9.3 [temp.expl.spec] paragraph 13,

> An explicit specialization of a function or variable template is inline only if it is declared with the `inline` specifier or defined as deleted, and independently of whether its function or variable template is inline.

The current wording does not specify the status of explicit specializations of `constexpr` and `consteval` function templates and members of class templates. Should a similar rule apply in those cases?

Proposed resolution (November, 2019):

Change 13.9.3 [temp.expl.spec] paragraph 14 as follows:

~~An explicit specialization of a function or variable template is inline only if it is declared with the `inline` specifier or defined as deleted, and independently of whether its function or variable template is inline.~~ **The inline, constexpr, and consteval properties of an explicit specialization of a function or variable template are determined by the explicit specialization and are independent of those properties of the template.** [*Example:*...

2441. Inline function parameters

Section: 9.2.7 [dcl.inline] **Status:** ready **Submitter:** Kristian Stasiowski **Date:** 2019-11-04

According to 9.2.7 [dcl.inline] paragraph 5,

> The `inline` specifier shall not appear on a block scope declaration.

Function parameters, however, are not block scope declarations, but they should also not be declared `inline`.

Proposed resolution (November, 2019):

Change 9.2.7 [dcl.inline] paragraph 5 as follows:

> The inline specifier shall not appear on a block scope declaration~~.~~[87] **or on the declaration of a function parameter.**